

[Date]

SWETA DUARI

Sweta Duari

Tops Technology

C programming assignment

Module 2 – Introduction to Programming Overview of C Programming

- **THEORY EXERCISE:** o Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today.

The History and Evolution of C Programming

C programming language has been a cornerstone of computer science for over four decades. Developed in the early 1970s by Dennis Ritchie at Bell Labs, C has undergone significant transformations, shaping the programming landscape and influencing numerous other languages.

Early Days

In 1969, Ken Thompson and Dennis Ritchie began working on the Unix operating system, which required a portable and efficient programming language. They developed the B programming language, which was later enhanced and renamed C. The first version of C, known as K&R C (after Kernighan and Ritchie), was released in 1972.

Evolution

Over the years, C has evolved through several standards:

1. K&R C (1972): The first version of C, characterized by its simplicity and lack of formal standardization.
2. ANSI C (1989): The American National Standards Institute (ANSI) standardized C, introducing features like function prototypes and void pointers.
3. C99 (1999): The C99 standard added features like variable-length arrays, designated initializers, and compound literals.
4. C11 (2011): The C11 standard introduced features like type-generic macros, anonymous structures, and improved Unicode support.

5. C17 (2018): The C17 standard clarified some aspects of the language and fixed some defects.

Importance

C's importance stems from its:

1. Portability: C code can be compiled on various platforms with minimal modifications.
2. Efficiency: C's low-level memory management and lack of runtime overhead make it suitable for systems programming.
3. Flexibility: C's versatility allows it to be used for a wide range of applications, from embedded systems to operating systems.
4. Influence: C has influenced many other programming languages, including C++, Java, and Python.

Why C is still used today

Despite the emergence of newer languages, C remains relevant due to:

1. Legacy systems: Many existing systems, including operating systems and embedded systems, are built using C.
2. Performance-critical code: C's efficiency makes it a popular choice for performance-critical code, such as device drivers and high-performance computing applications.
3. Systems programming: C's low-level memory management and lack of runtime overhead make it suitable for systems programming.
4. Education: C's simplicity and lack of abstractions make it an excellent teaching language for introductory programming courses.

Conclusion

In conclusion, C programming language has come a long way since its inception in the 1970s. Its evolution has been shaped by the needs of the programming community, and its importance lies in its portability, efficiency, flexibility, and influence on other languages. Despite the emergence of newer languages, C remains a fundamental language in computer science, and its continued use in various domains ensures its relevance for years to come.

• LAB EXERCISE: o Research and provide three real-world applications where C programming is extensively used, such as in embedded systems, operating systems, or game development:

Real-World Applications of C Programming

C programming is extensively used in various domains due to its efficiency, portability, and flexibility. Here are three real-world applications where C is widely used:

1. Embedded Systems

C is widely used in embedded systems, which are specialized computers used in devices like:

- Microcontrollers: C is used to program microcontrollers, which are small computers used in devices like traffic lights, appliances, and automotive systems.
- Automotive Systems: C is used in the development of automotive systems, such as engine control units, anti-lock braking systems (ABS), and airbag control units.
- Medical Devices: C is used in the development of medical devices, such as pacemakers, insulin pumps, and portable defibrillators.

Example: The Arduino platform, a popular platform for building embedded systems, uses C as its primary programming language.

2. Operating Systems

C is widely used in the development of operating systems, including:

- Linux Kernel: The Linux kernel is written primarily in C, with some assembly code.
- Windows Kernel: The Windows kernel is also written in C, with some C++ and assembly code.
- Real-Time Operating Systems (RTOS): C is used in the development of RTOS, which are used in applications that require predictable and fast responses to events.

Example: The Linux operating system, widely used in servers, desktops, and embedded systems, relies heavily on C programming.

3. Game Development

C is used in game development, particularly in:

- Game Engines: Some game engines, like the id Tech engine, use C as a primary programming language.
- Console Games: C is used in the development of console games, particularly for older consoles like the PlayStation 1 and Nintendo 64.
- Performance-Critical Code: C is used for performance-critical code in games, such as physics engines and graphics rendering.

Example: The classic game "Doom" was written in C, and its source code is still available today.

Why C is used in these applications

C's efficiency, portability, and flexibility make it an ideal choice for these applications. Its lack of runtime overhead and direct access to hardware resources make it suitable for systems programming and performance-critical code.

Conclusion

C programming is widely used in various domains, including embedded systems, operating systems, and game development. Its efficiency, portability, and flexibility make it an ideal choice for applications that require direct access to hardware resources and performance-critical code.

2. Setting Up Environment

Installing a C Compiler and Setting up an Integrated Development Environment (IDE)

Here's a step-by-step guide to installing a C compiler and setting up an IDE:

Installing GCC (GNU Compiler Collection)

1. Windows:

- Download the MinGW installer from the official website.
- Run the installer and select the GCC compiler.
- Follow the installation instructions to install MinGW.
- Add the MinGW bin directory to the system PATH environment variable.

2. macOS (using Homebrew):

- Install Homebrew if you haven't already.
- Run the command `brew install gcc` in the terminal.

3. Linux:

- Run the command `sudo apt-get install gcc` (for Ubuntu-based distributions) or `sudo yum install gcc` (for RPM-based distributions).

Setting up an IDE

1. Choose an IDE: Popular choices include:

- Code::Blocks
- Eclipse
- NetBeans
- Visual Studio Code

2. Install the IDE:

- Download the IDE installer from the official website.
- Follow the installation instructions to install the IDE.

3. Configure the IDE:

- Create a new project in the IDE.
- Select the C compiler (e.g., GCC) as the project's compiler.
- Configure the project's build settings and compiler options as needed.

Setting up the IDE with GCC

1. Code::Blocks:

- Create a new project and select "Console Application" as the project type.
- Select "GNU GCC Compiler" as the compiler.
- Configure the project's build settings and compiler options as needed.

2. Visual Studio Code:

- Install the "C/C++" extension.
- Create a new file with a .c extension.
- Configure the tasks.json file to use GCC as the compiler.

Verifying the Setup

1. Write a simple C program: Create a new file with a .c extension and write a simple "Hello, World!" program.
2. Compile and run the program: Use the IDE's build and run features to compile and run the program.
3. Verify the output: Check that the program outputs "Hello, World!" to the console.

• LAB EXERCISE: o Install a C compiler on your system and configure the IDE. Write your first program to print "Hello, World!" and run it.

By following these steps, you can install a C compiler and set up an IDE to start writing and compiling C programs.

Installing a C Compiler and Writing the First Program

Here's a step-by-step guide to installing a C compiler and writing the first program:

Installing GCC (GNU Compiler Collection)

1. Windows:

- Download and install MinGW from the official website.
- Follow the installation instructions to install MinGW.
- Add the MinGW bin directory to the system PATH environment variable.

2. macOS (using Homebrew):

- Install Homebrew if you haven't already.
- Run the command `brew install gcc` in the terminal.

3. Linux:

- Run the command `sudo apt-get install gcc` (for Ubuntu-based distributions) or `sudo yum install gcc` (for RPM-based distributions).

Configuring the IDE

1. Choose an IDE: For this example, we'll use Code::Blocks.
2. Install Code::Blocks:
 - Download the Code::Blocks installer from the official website.
 - Follow the installation instructions to install Code::Blocks.
3. Configure Code::Blocks:
 - Launch Code::Blocks and create a new project.
 - Select "Console Application" as the project type.
 - Select "GNU GCC Compiler" as the compiler.

Writing the First Program

1. Create a new file: In Code::Blocks, create a new file called `hello.c`.
2. Write the program: Write the following code in `hello.c`:

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello, World!\n");
```

```
    return 0;  
}
```

3. Save the file: Save the hello.c file.

Compiling and Running the Program

1. Build the project: In Code::Blocks, click on "Build" > "Build" or press F9 to build the project.
2. Run the program: Click on "Build" > "Run" or press Ctrl + F10 to run the program.
3. Verify the output: The program should output "Hello, World!" to the console.

Output Hello, World!

3. Basic Structure of a C Program

- **THEORY EXERCISE:** o Explain the basic structure of a C program, including headers, main function, comments, data types, and variables. Provide examples.

Basic Structure of a C Program

A C program consists of several key elements:

1. Headers

- Headers are files that contain function declarations and macro definitions.
- They are included at the beginning of a C program using the `#include` directive.
- Example: `#include <stdio.h>` includes the standard input/output header file.

2. Main Function

- The main function is the entry point of a C program.
- It is where program execution begins.
- Example: `int main() { ... }` declares the main function.

3. Comments

- Comments are used to document code and explain its purpose.
- There are two types of comments in C: single-line comments (`//`) and multi-line comments (`/* */`).
- Example: `// This is a single-line comment` or `/* This is a multi-line comment
*/`

4. Data Types

- Data types determine the type of value a variable can hold.
- C has several built-in data types, including:
 - int: whole numbers (e.g., 1, 2, 3)
 - float: decimal numbers (e.g., 3.14, -0.5)
 - char: single characters (e.g., 'a', 'B')
 - double: double-precision decimal numbers
- Example: `int x = 5;` declares an int variable x with value 5.

5. Variables

- Variables are used to store and manipulate data.
- A variable declaration consists of a data type and a variable name.
- Example: `int x;` declares an int variable x.

Example Program

Here's an example program that demonstrates these concepts:

```
#include <stdio.h>
```

```
// This is a single-line comment
```

```
/* This is a multi-line comment  
that spans multiple lines */
```

```
int main() {
```

```
    int x = 5; // declare and initialize an int variable
```

```
float pi = 3.14; // declare and initialize a float variable
char letter = 'A'; // declare and initialize a char variable

printf("The value of x is: %d\n", x);
printf("The value of pi is: %f\n", pi);
printf("The letter is: %c\n", letter);

return 0;
}
```

This program includes the `stdio.h` header file, declares the main function, uses comments to document the code, declares and initializes variables of different data types, and prints their values to the console.

Output

The value of x is: 5

The value of pi is: 3.140000

The letter is: A

This example demonstrates the basic structure of a C program, including headers, the main function, comments, data types, and variables.

- LAB EXERCISE: o Write a C program that includes variables, constants, and comments. Declare and use different data types (int, char, float) and display their values.

C Program with Variables, Constants, and Comments

Here's an example program that demonstrates the use of variables, constants, and comments in C:

```
#include <stdio.h>

// Define a constant
#define PI 3.14

int main() {
    // Declare and initialize variables
    int age = 25;
    char initial = 'J';
    float height = 5.9;

    // Display the values of variables
    printf("Age: %d\n", age);
    printf("Initial: %c\n", initial);
    printf("Height: %.1f feet\n", height);

    // Use the constant
    printf("Value of PI: %.2f\n", PI);

    // Declare and initialize another variable
    int year = 2024;
    printf("Current year: %d\n", year);

    return 0;
```

}

Output

Age: 25

Initial: J

Height: 5.9 feet

Value of PI: 3.14

Current year: 2024

4. Operators in C

- THEORY EXERCISE: o Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators.

Operators in C

C provides various operators to perform different operations on variables and values. Here are notes explaining each type of operator:

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, and division.

- + (addition)
- - (subtraction)
- * (multiplication)
- / (division)
- % (modulus or remainder)

Example: `a = 5 + 3;` (adds 5 and 3 and assigns the result to a)

2. Relational Operators

Relational operators are used to compare two values and return a boolean result (true or false).

- == (equal to)
- != (not equal to)
- > (greater than)
- < (less than)
- >= (greater than or equal to)
- <= (less than or equal to)

Example: `if (a > 5) { ... }` (checks if a is greater than 5)

3. Logical Operators

Logical operators are used to combine relational operators and return a boolean result.

- && (logical AND)
- || (logical OR)
- ! (logical NOT)

Example: `if (a > 5 && b < 10) { ... }` (checks if a is greater than 5 and b is less than 10)

4. Assignment Operators

Assignment operators are used to assign a value to a variable.

- = (simple assignment)
- += (addition assignment)
- -= (subtraction assignment)
- *= (multiplication assignment)
- /= (division assignment)
- %= (modulus assignment)

Example: `a += 5;` (adds 5 to a and assigns the result back to a)

5. Increment/Decrement Operators

Increment and decrement operators are used to increase or decrease the value of a variable by 1.

- ++ (increment)
- -- (decrement)

Example: `a++;` (increments a by 1)

6. Bitwise Operators

Bitwise operators are used to perform operations on the binary representation of numbers.

- & (bitwise AND)
- | (bitwise OR)
- ^ (bitwise XOR)
- ~ (bitwise NOT)
- << (left shift)
- >> (right shift)

Example: `a = 5 & 3;` (performs a bitwise AND operation on 5 and 3)

7. Conditional Operator

The conditional operator is used to evaluate a condition and return one of two values.

- ?: (ternary operator)

Example: `a = (b > 5) ? 10 : 20;` (assigns 10 to a if b is greater than 5, otherwise assigns 20)

These operators are used to perform various operations in C programming and are essential for writing efficient and effective code.

• **LAB EXERCISE:** o Write a C program that accepts two integers from the user and performs arithmetic, relational, and logical operations on them. Display the results.

C Program for Arithmetic, Relational, and Logical Operations

Here's an example program that accepts two integers from the user and performs arithmetic, relational, and logical operations on them:

```
#include <stdio.h>

int main() {
    int num1, num2;
```

```
// Get input from user
printf("Enter first integer: ");
scanf("%d", &num1);
printf("Enter second integer: ");
scanf("%d", &num2);

// Arithmetic operations
printf("\nArithmetic Operations:\n");
printf("Addition: %d + %d = %d\n", num1, num2, num1 + num2);
printf("Subtraction: %d - %d = %d\n", num1, num2, num1 - num2);
printf("Multiplication: %d * %d = %d\n", num1, num2, num1 * num2);
if (num2 != 0) {
    printf("Division: %d / %d = %.2f\n", num1, num2, (float)num1 / num2);
} else {
    printf("Error: Division by zero is not allowed.\n");
}
printf("Modulus: %d %% %d = %d\n", num1, num2, num1 % num2);

// Relational operations
printf("\nRelational Operations:\n");
printf("%d == %d: %d\n", num1, num2, num1 == num2);
printf("%d != %d: %d\n", num1, num2, num1 != num2);
printf("%d > %d: %d\n", num1, num2, num1 > num2);
printf("%d < %d: %d\n", num1, num2, num1 < num2);
printf("%d >= %d: %d\n", num1, num2, num1 >= num2);
```

```
printf("%d <= %d: %d\n", num1, num2, num1 <= num2);

// Logical operations
printf("\nLogical Operations:\n");
printf("%d > 0 && %d > 0: %d\n", num1, num2, num1 > 0 && num2 > 0);
printf("%d > 0 || %d > 0: %d\n", num1, num2, num1 > 0 || num2 > 0);
printf("!%d: %d\n", num1, !num1);

return 0;
}
```

Output

The output will vary based on the input provided by the user. Here's an example output:

Enter first integer: 10

Enter second integer: 5

Arithmetic Operations:

Addition: $10 + 5 = 15$

Subtraction: $10 - 5 = 5$

Multiplication: $10 * 5 = 50$

Division: $10 / 5 = 2.00$

Modulus: $10 \% 5 = 0$

Relational Operations:

$10 == 5$: 0

$10 != 5$: 1

$10 > 5$: 1

$10 < 5$: 0

$10 >= 5$: 1

$10 <= 5$: 0

Logical Operations:

$10 > 0 \ \&\& \ 5 > 0$: 1

$10 > 0 \ || \ 5 > 0$: 1

$!10$: 0

This program demonstrates the use of arithmetic, relational, and logical operators in C programming.

5. Control Flow Statements in C

- **THEORY EXERCISE:** Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.

Decision-Making Statements in C

Decision-making statements are used to control the flow of a program based on certain conditions. C provides several decision-making statements:

1. If Statement

The if statement is used to execute a block of code if a certain condition is true.

Syntax:

```
if (condition) {  
    // code to be executed  
}
```

Example:

```
int x = 5;  
if (x > 0) {  
    printf("x is positive\n");  
}
```

2. If-Else Statement

The if-else statement is used to execute one block of code if a certain condition is true and another block of code if the condition is false.

Syntax:

```
if (condition) {  
    // code to be executed if condition is true  
} else {  
    // code to be executed if condition is false  
}
```

Example:

```
int x = -5;
if (x > 0) {
    printf("x is positive\n");
} else {
    printf("x is negative or zero\n");
}
```

3. Nested If-Else Statement

A nested if-else statement is used to check multiple conditions and execute different blocks of code based on those conditions.

Syntax:

```
if (condition1) {
    if (condition2) {
        // code to be executed if both conditions are true
    } else {
        // code to be executed if condition1 is true and condition2 is false
    }
} else {
    // code to be executed if condition1 is false
}
```

Example:

```
int x = 5;
int y = 10;
if (x > 0) {
```

```
if (y > x) {  
    printf("y is greater than x\n");  
} else {  
    printf("y is not greater than x\n");  
}  
  
} else {  
    printf("x is not positive\n");  
}  
}
```

4. Switch Statement

The switch statement is used to execute different blocks of code based on the value of a variable or expression.

Syntax:

```
switch (expression) {  
    case value1:  
        // code to be executed if expression equals value1  
        break;  
    case value2:  
        // code to be executed if expression equals value2  
        break;  
    default:  
        // code to be executed if expression does not equal any value  
        break;  
}
```

Example:

```
int day = 2;
switch (day) {
    case 1:
        printf("Monday\n");
        break;
    case 2:
        printf("Tuesday\n");
        break;
    case 3:
        printf("Wednesday\n");
        break;
    default:
        printf("Invalid day\n");
        break;
}
```

- LAB EXERCISE: o Write a C program to check if a number is even or odd using an if-else statement. Extend the program using a switch statement to display the month name based on the user's input (1 for January, 2 for February, etc.).

C Program to Check Even/Odd and Display Month Name

Here's an example program that checks if a number is even or odd using an if-else statement and displays the month name based on the user's input using a switch statement:

```
#include <stdio.h>
```

```
int main() {  
    int num, month;  
    // Check if a number is even or odd  
    printf("Enter a number: ");  
    scanf("%d", &num);  
    if (num % 2 == 0) {  
        printf("%d is even.\n", num);  
    } else {  
        printf("%d is odd.\n", num);  
    }  
    // Display month name based on user's input  
    printf("Enter month number (1-12): ");  
    scanf("%d", &month);  
    switch (month) {  
        case 1:  
            printf("January\n");  
            break;  
        case 2:  
            printf("February\n");  
            break;  
        case 3:  
            printf("March\n");  
            break;  
        case 4:  
            printf("April\n");  
            break;  
    }  
}
```

case 5:

```
printf("May\n");
```

```
break;
```

case 6:

```
printf("June\n");
```

```
break;
```

case 7:

```
printf("July\n");
```

```
break;
```

case 8:

```
printf("August\n");
```

```
break;
```

case 9:

```
printf("September\n");
```

```
break;
```

case 10:

```
printf("October\n");
```

```
break;
```

case 11:

```
printf("November\n");
```

```
break;
```

case 12:

```
printf("December\n");
```

```
break;
```

default:

```
printf("Invalid month number.\n");
```

```
        break;  
    }
```

```
    return 0;  
}
```

Output

Enter a number: 10

10 is even.

Enter month number (1-12): 5

May

6.Looping in C

THEORY EXERCISE: o Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

Loops Comparison

Loops are essential in programming, allowing you to execute a block of code repeatedly. Here's a comparison of while loops, for loops, and do-while loops:

While Loops

- Entry-controlled loop: checks the condition before executing the loop body
- Zero or more iterations: suitable when the number of iterations is unknown
- Use cases ^{1 2}:
 - Unknown iteration counts
 - User input loops
 - Validation checks

For Loops

- Initialization, condition, and increment/decrement: allows for precise control
- Known number of iterations: ideal when you know exactly how many times the loop will iterate
- Use cases ³:
 - Iterating over arrays or collections
 - Performing repetitive tasks with a fixed number of iterations

Do-While Loops

- Exit-controlled loop: executes the loop body at least once before checking the condition
- One or more iterations: suitable when you want to guarantee at least one execution
- Use cases :
 - User menu loops

- User prompt loops
- Repeat until success scenarios

Choosing the Right Loop

When deciding which loop to use, consider the following factors ³:

- Number of iterations: If you know the exact number, use a for loop. If it's unknown, use a while loop.
- Condition checking: If you want to check the condition before executing the loop body, use a while loop. If you want to execute the loop body at least once, use a do-while loop.
- Initialization: If you need to initialize variables before the loop, use a for loop or while loop.

• LAB EXERCISE: o Write a C program to print numbers from 1 to 10 using all three types of loops (while, for, do-while).

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    // Using while loop
```

```
    printf("Numbers from 1 to 10 using while loop:\n");
```

```
    i = 1;
```

```
    while (i <= 10) {
```

```
        printf("%d ", i);
```

```
        i++;
```

```
    }
```

```
    printf("\n");
```

```
    // Using for loop
```

```
    printf("Numbers from 1 to 10 using for loop:\n");
```

```
for (i = 1; i <= 10; i++) {  
    printf("%d ", i);  
}  
printf("\n");  
  
// Using do-while loop  
printf("Numbers from 1 to 10 using do-while loop:\n");  
i = 1;  
do {  
    printf("%d ", i);  
    i++;  
} while (i <= 10);  
printf("\n");  
  
return 0;  
}
```

Output:

Numbers from 1 to 10 using while loop:

1 2 3 4 5 6 7 8 9 10

Numbers from 1 to 10 using for loop:

1 2 3 4 5 6 7 8 9 10

Numbers from 1 to 10 using do-while loop:

1 2 3 4 5 6 7 8 9 10

Explanation:

- We use a while loop to print numbers from 1 to 10 by initializing a variable i to 1 and incrementing it until i is less than or equal to 10.
- A for loop is used to achieve the same result, but with a more concise syntax that combines initialization, condition checking, and increment in a single line.
- The do-while loop is similar to the while loop but ensures that the loop body is executed at least once before c

7. Loop Control Statements

- **THEORY EXERCISE:** o Explain the use of break, continue, and goto statements in C. Provide examples of each.

Control Flow Statements in C: Break, Continue, and Goto

In C programming, control flow statements are used to manage the flow of a program's execution. Here are explanations of break, continue, and goto statements, along with examples:

Break Statement

The break statement terminates the execution of a loop or switch statement immediately. It's like an emergency exit button that stops the loop and continues executing the code after the loop.

Example:

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        break;  
    }  
    printf("%d ", i);  
}
```

// Output: 1 2 3 4

In this example, the loop breaks when i equals 5, and the program continues executing the code after the loop ¹.

Continue Statement

The continue statement skips the current iteration of a loop and moves to the next iteration. It's like a "skip" button that bypasses the remaining statements in the current iteration.

Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
    printf("%d ", i);  
}  
// Output: 1 2 4 5
```

In this example, the continue statement skips the iteration when i equals 3, and the program prints the numbers 1, 2, 4, and 5 ².

Goto Statement

The goto statement transfers control to a labeled statement in the program. It's like a teleportation device that jumps to a specific point in the code.

Example:

```
int age = 20;  
if (age >= 18) {  
    goto adult;  
}
```

```
printf("You are not an adult.\n");
```

```
goto end;
```

```
adult:
```

```
    printf("You are an adult.\n");
```

```
end:
```

```
    return 0;
```

```
// Output: You are an adult.
```

• **LAB EXERCISE:** o Write a C program that uses the break statement to stop printing numbers when it reaches 5. Modify the program to skip printing the number 3 using the continue statement.

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    printf("Printing numbers from 1 to 10, stopping at 5:\n");
```

```
    for (i = 1; i <= 10; i++) {
```

```
        if (i == 5) {
```

```
            break; // Stop printing numbers when i reaches 5
```

```
        }
```

```
        printf("%d ", i);
```

```
    }
```

```
    printf("\n");
```

```
    printf("Printing numbers from 1 to 5, skipping 3:\n");
```

```
    for (i = 1; i <= 5; i++) {
```

```
        if (i == 3) {
```

```
            continue; // Skip printing the number 3
```

```
        }
```

```
        printf("%d ", i);  
    }  
    printf("\n");  
  
    return 0;  
}
```

8.Functions in C

- **THEORY EXERCISE:** o What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.

Functions in C

Functions are blocks of code that perform a specific task. They are reusable, modular, and help to organize code, making it more efficient and easier to maintain.

Function Declaration

A function declaration, also known as a function prototype, specifies the function's name, return type, and parameters. It's like a contract that defines what the function will do.

Example:

```
int add(int a, int b);
```

This declaration specifies a function named `add` that takes two `int` parameters and returns an `int` value.

Function Definition

A function definition provides the actual implementation of the function. It includes the function body, which contains the code that performs the task.

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

This definition implements the `add` function, which takes two `int` parameters and returns their sum.

Calling a Function

To use a function, you need to call it by its name and pass the required arguments. The function will execute its code and return the result, if any.

Example:

```
int result = add(5, 3);
```

```
printf("%d\n", result); // Output: 8
```

In this example, the add function is called with arguments 5 and 3, and the result is stored in the result variable.

Printing numbers from 1 to 10, stopping at 5:

```
1 2 3 4
```

Printing numbers from 1 to 5, skipping 3:

```
1 2 4 5
```

• **LAB EXERCISE:** o Write a C program that calculates the factorial of a number using a function. Include function declaration, definition, and call.

Here is a C program that calculates the factorial of a number using a function:

```
#include <stdio.h>
```

```
int factorial(int n);
```

```
void main()
```

```
{
```

```
    int num, ans;
```

```
    printf("Enter Any Number: ");
```

```
    scanf("%d", &num);
```

```
    ans = factorial(num);
```

```
    printf("\nFactorial : %d", ans);
```

```
}
```

```
int factorial(int n)
```

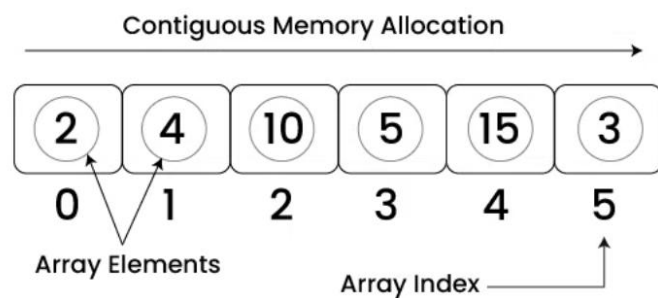
```
{  
  
    int ans = 1, i = n;  
  
    while (i > 0)  
  
    {  
  
        ans = ans * i;  
  
        i--;  
  
    }  
  
    return ans;  
  
}
```

9.Arrays in c

- **THEORY EXERCISE:** o Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

Array is a linear data structure where all elements are arranged sequentially. It is a collection of elements of **same data type** stored at **contiguous memory locations**.

What is
Array
Data Structure



With an array, you can:

- Store a collection of numbers, words, or objects.
- Access any value using its index (position).
- Read, update, insert, or remove any of the array values.

Declaring Arrays

The general syntax for declaring an array in C is:

```
type array_name[size];
```

Example:

```
int scores[5];
```

This declares an array scores that can hold 5 integer values.

Initializing Arrays

You can initialize an array when declaring it:

```
int scores[5] = {90, 85, 95, 88, 92};
```

Accessing Array Elements

You can access each element using its index:

```
scores[0] = 90;
```

- One-Dimensional Arrays:

A one-dimensional array is a linear sequence of elements. It looks like a list of values.

Example:

```
int numbers[5];
```

```
numbers = 10;
```

```
numbers = 20;
```

```
numbers = 30;
```

```
numbers = 40;
```

```
numbers = 50;
```

```
printf("%d", numbers);
```

```
// Output: 30
```

Here, numbers is a one-dimensional array with 5 elements indexed from 0 to 4.

- Multi-Dimensional Arrays:

Multi-dimensional arrays are arrays of arrays. The most common type is the two-dimensional array, which can be thought of as a table or matrix with rows and columns.

Example of a Two-Dimensional Array:

```
int matrix;
```

```
matrix = 1;
```

```
matrix = 2;
```

```
matrix = 3;
```

```
matrix = 12;
```

```
printf("%d", matrix);
```

```
// Output: 3
```

This matrix has 3 rows and 4 columns. You access elements using two indices: the row index and the column index.

Differentiate between one-dimensional and multi-dimensional arrays with examples.

	One dimensional	Multi-dimensional
Definition	A linear collection of elements stored in contiguous memory.	A collection of arrays arranged in rows, columns, or higher dimensions.
Syntax	data_type array_name[size];	data_type array_name[size1][size2]...[sizeN];
Example	int arr[5];	int arr[3][4]; (2D) or int arr[2][3][4]; (3D)
Accessing Elements	Accessed using a single index: arr[i].	Accessed using multiple indices: arr[i][j] (2D), arr[i][j][k] (3D).
Use Case	Suitable for linear	Suitable for tabular data (2D), grids,

	data like lists, sequences, or vectors	or layered structures (3D).
--	--	-----------------------------

- LAB EXERCISE: o Write a C program that stores 5 integers in a one-dimensional array and prints them. Extend this to handle a two-dimensional array (3x3 matrix) and calculate the sum of all elements.

```
#include <stdio.h>
#include<conio.h>

int main()
{

    int arr[5];
    int i;

    printf("\nEnter 5 integers : ");
    for (i = 0; i < 5; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("\nYou entered (1D Array) : ");
    for (i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");

    int matrix[3][3];
    int sum = 0, r, c;

    printf("\nEnter values : ");
```

```
for (r= 0; r < 3; r++)
{
    for (c= 0; c< 3; c++)
    {
        scanf("%d", &matrix[r][c]);
        sum += matrix[r][c];
    }
}

printf("\nur matrix : ");
for (r = 0; r < 3; r++)
{
    for (c = 0; c < 3; c++)
    {
        printf(" %d ", matrix[r][c]);
    }
    printf("\n");
}

printf("\nSum of all numbers in matrix is = %d ", sum);

return 0;
}
```

10. Pointers in C

• **THEORY EXERCISE:** o Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?

- What are Pointers in C?

- Pointers in C are special variables that store the memory address of another variable, rather than its direct value. This means that a pointer “points” to the location in memory where data is stored. For example, while an int variable might store the value 10, an int pointer stores the address of where that integer is kept.

- Declaration and Initialization of Pointers:

- Declaration-

A pointer is declared by specifying the data type it will point to, followed by an asterisk (*), and then the pointer’s name.

Example:

```
int *ptr;    // pointer to an integer
char *ch_ptr; // pointer to a character
float *f_ptr; // pointer to a float
```

- Initialization-

To initialize a pointer, assign it the memory address of a variable using the address-of operator &.

Example:

```
int num = 25;  
int *ptr = &num; // ptr now holds the address of num
```

You can also assign NULL to a pointer if it doesn't point to any variable yet:

```
int *ptr = NULL; // safe initialization with no address
```

- Why are Pointers Important in C?
 - Direct Memory Access: Pointers allow you to access and modify memory locations directly, providing more control over how data is stored and managed.
 - Efficient Data Handling: Arrays, strings, and complex data structures like linked lists rely on pointers for efficient manipulation and access.
 - Dynamic Memory Allocation: Functions like `malloc()` and `free()` use pointers to allocate and release memory at runtime.
 - Function Arguments: Pointers enable functions to modify the values of variables passed to them, making it possible to use pass-by-reference.
 - Building Data Structures: Advanced structures such as linked lists, trees, and graphs are constructed using pointers to create dynamic relationships between elements.

- LAB EXERCISE: o Write a C program to demonstrate pointer usage. Use a pointer to modify the value of a variable and print the result.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 11;
```

```
    int *ptr = &a;
```

```
    printf("\nBefore modification num = %d ", a);
```

```
    *ptr = 4;
```

```
    printf("\nAfter modification num = %d ", a);
```

```
    return 0;
```

```
}
```

11. Strings in C

- **THEORY EXERCISE:** o Explain string handling functions like `strlen()`, `strcpy()`, `strcat()`, `strcmp()`, and `strchr()`. Provide examples of when these functions are useful.

- strings are arrays of characters terminated by a null character (`'\0'`). Several standard library functions help manipulate and work with strings efficiently

- `strlen()`:

- Purpose: Returns the length of a string, excluding the terminating null character.

- Usefulness: When you need to know how many characters a string contains.

Example:

If you want to print the length of a user's name or check if a string is empty.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main()
```

```
{
```

```
    char ch[] = "android";
```

```
    printf("%d",strlen(ch));
```

```
    return 0;
```

```
}
```

//output=7

- strcpy():

➤ Purpose: Copies one string into another.

➤ Usefulness: To assign or duplicate string content.

Example:

Copying a source string into a destination buffer.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main()
```

```
{
```

```
    char ch[50];
```

```
    char ch2[50];
```

```
    printf("Enter Your Name: ");
```

```
    scanf("%s",ch);
```

```
    strcpy(ch2,ch);
```

```
    printf("Your Name is : %s",ch2);
```

```
    return 0;
```

```
}
```

//output=

Enter Your Name: harsh

Your Name is : harsh

- strcat():

- Purpose: appends one string to the end of another.
- Usefulness: To build or combine strings dynamically.

Example:

```
#include<stdio.h>
#include<string.h>

int main()
{
    char fname[]="tops";
    char lname[]="tech";

    printf("%s",strcat(fname,lname));

    return 0;
}
//output=
topstech
```

- strcmp():

- Purpose: Compares two strings lexicographically.
- Usefulness: To check if strings are equal or to sort strings alphabetically.

Example:

```
#include<stdio.h>
#include<string.h>

int main()
{
    char ch[]="mango";
    char ch2[50];

    do
    {
        printf("Fav. Fruit?");
        scanf("%s",ch2);
    }
    while(strcmp(ch,ch2)!=0);

    printf("Answer is Correct");

    return 0;
}
```

```
//output=
Fav. Fruit?mango
Answer is Correct
```

- strchr():

- Purpose: Finds the first occurrence of a character in a string.
- Usefulness: To locate a character inside a string.

➤ e.g., finding '@' in an email address.

Example:

Searching for a character in a string.

```
char email[] = "user@example.com";
char *ptr = strchr(email, '@');
if(ptr != NULL)
{
    printf("Found '@' at position: %ld\n", ptr - email);
}
else
{
    printf("'@' not found\n");
}
```

- LAB EXERCISE: o Write a C program that takes two strings from the user and concatenates them using strcat(). Display the concatenated string and its length using strlen().

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[10], str2[10];

    printf("Enter the first string: ");
    scanf("%s", str1);

    printf("Enter the second string: ");
    scanf("%s", str2);

    strcat(str1, str2);

    printf("\nConcatenated String: %s", str1);

    printf("\nLength of concatenated string: %d", strlen(str1));

    return 0;
}
```

12. Structures in C

- **THEORY EXERCISE:** o Explain the concept of structures in C. Describe how to declare, initialize, and access structure members.

Structures in C

In C, a structure (also known as a struct) is a user-defined data type that allows you to combine multiple variables of different data types into a single unit. Structures are useful for representing complex data structures, such as records or objects.

- Declaring Structures

The general syntax for declaring a structure in C is:

```
struct structure_name {  
    data_type member1;  
    data_type member2;  
    ...  
    data_type memberN;  
};
```

Example:

```
struct Student {  
    int roll_number;  
    char name[100];  
    float marks;
```

```
};
```

This declares a structure Student with three members: roll_number, name, and marks.

- Initializing Structures

You can initialize a structure variable when declaring it:

```
struct Student student1 = {1, "John Doe", 85.5};
```

Alternatively, you can use designated initializers (C99 and later):

```
struct Student student2 = {  
    .roll_number = 2,  
    .name = "Jane Doe",  
    .marks = 90.0  
};
```

- Accessing Structure Members

You can access structure members using the dot (.) operator:

```
student1.roll_number = 1;  
printf("%d", student1.roll_number); // Output: 1
```

If you have a pointer to a structure, you can use the arrow (->) operator:

```
struct Student *ptr = &student1;  
printf("%d", ptr->roll_number); // Output: 1
```

Example Program

Here's an example program that demonstrates structure operations:

```
#include <stdio.h>
```

```
struct Student {  
    int roll_number;  
    char name[100];  
    float marks;  
};
```

```
int main() {  
    struct Student student1 = {1, "John Doe", 85.5};  
  
    // Accessing structure members  
    printf("Roll Number: %d\n", student1.roll_number);  
    printf("Name: %s\n", student1.name);  
    printf("Marks: %.2f\n", student1.marks);  
  
    // Modifying structure members  
    student1.marks = 90.0;  
    printf("Updated Marks: %.2f\n", student1.marks);  
}
```

```
    return 0;
}
```

• LAB EXERCISE: o Write a C program that defines a structure to store a student's details (name, roll number, and marks). Use an array of structures to store details of 3 students and print them.

```
#include <stdio.h>
#include<string.h>
```

```
struct Student {
    char name[50];
    int roll;
    float marks;
}s[3];
```

```
int main()
{
```

```
    int i;
```

```
    for (i = 0; i < 3; i++)
    {
```

```
        printf("\nEnter Name : ");
        scanf("%s", s[i].name);
```

```
        printf("\nEnter Roll Number : ");
        scanf("%d", &s[i].roll);
```

```
        printf("\nEnter Marks : ");
        scanf("%f", &s[i].marks);
```

```
}
```

```
for (i = 0; i < 3; i++)
```

```
{
```

```
    printf("\nName : %s ", s[i].name);
```

```
    printf("\nRoll No : %d", s[i].roll);
```

```
    printf("\nMarks : %f", s[i].marks);
```

```
}
```

```
return 0;
```

```
}
```

13. File Handling in C

• **THEORY EXERCISE:** o Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files.

- Importance of File Handling in C:

- File handling in C is important because it allows a program to store data permanently by saving it to files. Unlike variables stored in memory, which lose their values after the program ends, files keep data safe on the disk.

- This helps in:

1. Saving and retrieving data between program runs.
2. Sharing data between different programs.
3. Handling large amounts of data beyond memory limits.
4. Reading data from input files and writing output files such as logs or reports.

- File Operations in C:

- Opening a File-

To open a file, use the `fopen()` function with the file name and mode (read, write, append):

```
// open file for reading
```

```
FILE *fp = fopen("filename.txt", "r");
```

Common modes include:

Mode	Purpose
"r"	Open file for reading
"w"	Open file for writing (creates or overwrites)
"a"	Open file for appending
"r+"	Open for reading and writing
"w+"	Open for writing and reading (overwrite)
"a+"	Open for appending and reading

If `fopen()` fails, it returns `NULL`.

➤ Closing a File-

-After file operations, close the file with:
`fclose(fp);`

➤ Reading from a File-

1. `fgetc(fp)` — reads a single character.

2. `fgets(buffer, size, fp)` — reads a line of text into a buffer.

3. `fread(ptr, size, count, fp)` — reads binary data.

➤ Writing to a File-

1. `fputc(ch, fp)` — writes a character.
2. `fputs(str, fp)` — writes a string.
3. `fprintf(fp, "format", ...)` — formatted write.
4. `fwrite(ptr, size, count, fp)` — writes binary data.

- **LAB EXERCISE:** o Write a C program to create a file, write a string into it, close the file, then open the file again to read and display its contents.

```
#include<stdio.h>
#include<string.h>

int main()
{
    FILE *s1;

    char name[]="Sweta";
    char c[100];

    s1 = fopen(" file.text","w");
    fprintf(h1,"%s",name);

    fclose(s1);

    s1 = fopen("file.text","r");
    fgets(c , 100 , s1);
    fclose(s1);

    printf("%s",c);

    return 0;

}
```

EXTRA LAB EXERCISES FOR IMPROVING PROGRAMMING LOGIC

1. Operators

LAB EXERCISE 1: Simple Calculator • Write a C program that acts as a simple calculator. The program should take two numbers and an operator as input from the user and perform the respective operation (addition, subtraction, multiplication, division, or modulus) using operators. • Challenge: Extend the program to handle invalid operator inputs.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    float n1,n2;
```

```
    int select;
```

```
    int a,b;
```

```
    printf("Enter first number : ");
```

```
    scanf("%f",&a);
```

```
    printf("\nEnter second number : ");
```

```
    scanf("%f",&b);
```

```
    printf("\n----- ");
```

```
    printf("\nEnter the operation : ");
```

```
    printf("\n 1. for addition(+)");
```

```
    printf("\n 2. for sub(-)");
```

```
    printf("\n 3. for multi(*)");
```

```
    printf("\n 4. for division(/)");
```

```
printf("\n5. for modules(%)\\n");  
scanf("%d",&select);
```

```
switch (select)  
{  
    case 1 :  
        printf("\\n %f + %f = %.2f ", a , b, a+b);  
        break;  
  
    case 2 :  
        printf("\\n %f - %f = %.2f ", a , b,a-b);  
        break;  
  
    case 3 :  
        printf("\\n %f * %f = %.2f ", a , b, a*b);  
        break;  
  
    case 4 :  
        printf("\\n %f / %f = %.2f ", a , b, a/b);  
        break;  
  
    case 5 :  
        printf("\\n %d %% %d = %d ", a , b, a % b );  
        break;  
    default:  
        printf("\\nInvalid operator");  
  
}
```

```
return 0;
```

LAB EXERCISE 2: Check Number Properties • Write a C program that takes an integer from the user and checks the following using different operators:

- o Whether the number is even or odd.
- o Whether the number is positive, negative, or zero.
- o Whether the number is a multiple of both 3 and 5.

```
#include<stdio.h>
```

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter a number : ");
```

```
    scanf("%d",&n);
```

```
    (n % 2 == 0)? printf("this is an even number") :printf("this is an odd number");
```

```
    (n > 0) ? printf("\n%d is positive", n) :(n < 0) ? printf("\n%d is negative", n) : printf("\n%d is zero", n);
```

```
    (n%3 == 0 && n%5 == 0) ? printf("\nthe number is a multiple of 3, 5") : printf("\nthe number is NOT a multiple of 3, 5");
```

```
    return 0;
```

2. Control Statements

LAB EXERCISE 1: Grade Calculator • Write a C program that takes the marks of a student as input and displays the corresponding grade based on the following conditions: o Marks > 90: Grade A o Marks > 75 and <= 90: Grade B o Marks > 50 and <= 75: Grade C o Marks <= 50: Grade D • Use if-else or switch statements for the decision-making process.

```
#include <stdio.h>
#include <conio.h>

int main()
{
    int marks;

    printf("Enter the students marks : ");
    scanf("%d", &marks);

    if (marks > 90)
    {
        printf("\nGrade : A");
    }
    else if (marks > 75 && marks <= 90)
    {
        printf("\nGrade : B");
    }
    else if (marks > 50 && marks <= 75)
    {
        printf("\nGrade : C");
    }
    else
```

```

{
    printf("\nGrade : D");
}

return 0;
}

```

LAB EXERCISE 2: Number Comparison • Write a C program that takes three numbers from the user and determines:

- o The largest number.
- o The smallest number.

• Challenge: Solve the problem using both if-else and switch-case statements.

```

#include<stdio.h>
#include<conio.h>

```

```

int main()
{
    int a,b,c,biggest,smallest;

    printf("Enter value for 1 num : ");
    scanf("%d",&a);
    printf("Enter value for 2 num : ");
    scanf("%d",&b);
    printf("Enter value for 3 num : ");
    scanf("%d",&c);

```

```

    biggest = (a >= b && a >= c) ? a : ( b >= a && b >= c) ? b :c;

```

```

    smallest = (a <= b && a <= c) ? a : ( b <= a && b <= c) ?b: c;

```

```

    switch(biggest)
    {
        case 0 :
            break;

```

```
    default:  
    printf("\nBiggest number is %d", biggest);  
}
```

```
switch(smallest)  
{  
    case 0 :  
        break;  
    default:  
        printf("\nSmallest number is %d", smallest);  
}
```

```
return 0;  
}
```

3. Loops

LAB EXERCISE 1: Prime Number Check • Write a C program that checks whether a given number is a prime number or not using a for loop. • Challenge: Modify the program to print all prime numbers between 1 and a given number.

```
#include <stdio.h>
```

```
// Recursive function to check if a number is prime
```

```
int isPrimeRecursive(int number, int divisor) {
```

```
    // Base case: If the number is less than or equal to 1, it's not prime
```

```
    if (number <= 1) {
```

```
        return 0;
```

```
    }
```

```
    // Base case: If divisor reaches 1, and no divisors were found, the  
    number is prime
```

```
    if (divisor == 1) {
```

```
        return 1;
```

```
    }
```

```
    // If the number is divisible by the current divisor, it's not prime
```

```
    if (number % divisor == 0) {
```

```
        return 0;
```

```
    }
```

```
    // Recursively call the function with divisor reduced by 1
```

```
    return isPrimeRecursive(number, divisor - 1);
```

```
}
```

```
int main() {
```

```
    int number;
```

```
    printf("Enter a number: ");
```

```
    scanf("%d", &number);
```

```

    // Start checking from number / 2 for efficiency (or sqrt(number)
for better optimization)
    if (isPrimeRecursive(number, number / 2)) {
        printf("%d is a prime number.\n", number);
    } else {
        printf("%d is not a prime number.\n", number);
    }
    return 0;
}

```

LAB EXERCISE 2: Multiplication Table • Write a C program that takes an integer input from the user and prints its multiplication table using a for loop. • Challenge: Allow the user to input the range of the multiplication table (e.g., from 1 to N).

```

include <stdio.h>

int main() {
    int num;

    printf("Enter an integer to print its multiplication table: ");
    scanf("%d", &num);
    printf("Enter the range of number");
    scanf("%d",&a);

    printf("Multiplication Table of %d:\n", num);

    for (int i = 1; i <= a; i++) {

        printf("%d x %d = %d\n", num, i, num * i);
    }

    return 0;
}

```

```
}
```

LAB EXERCISE 3: Sum of Digits • Write a C program that takes an integer from the user and calculates the sum of its digits using a while loop. • Challenge: Extend the program to reverse the digits of the number.

```
#include <stdio.h>
#include <conio.h>

int main() {
    int number, sum = 0, digit;

    printf("Enter a number: ");
    scanf("%d", &number);

    while (number != 0) {
        digit = number % 10;
        reversedNum = reversedNum * 10 + digit;
        sum += digit;
        number /= 10;
    }

    printf("Sum of the individual digits: %d\n", sum);
    printf("The reversed number is %d", reversedNum);

    return 0;
}
```

4. Arrays

LAB EXERCISE 1: Maximum and Minimum in Array • Write a C program that accepts 10 integers from the user and stores them in an array. The program should then find and print the maximum and minimum values in the array. • Challenge: Extend the program to sort the array in ascending order.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main() {
```

```
    int arr[10];
```

```
    int max, min, temp;
```

```
    // Input 10 integers
```

```
    printf("Enter 10 integers:\n");
```

```
    for (int i = 0; i < 10; i++) {
```

```
        printf("Element %d: ", i + 1);
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    // Initialize max and min with the first element
```

```
    max = arr[0];
```

```
    min = arr[0];
```

```
    // Find max and min
```

```
    for (int i = 1; i < 10; i++) {
```

```
        if (arr[i] > max) {
```

```
            max = arr[i];
```

```
        }
```

```
        if (arr[i] < min) {
```

```
        min = arr[i];
    }
}
```

```
printf("Maximum value: %d\n", max);
printf("Minimum value: %d\n", min);
```

```
// Challenge: Sort the array in ascending order using Bubble Sort
```

```
for (int i = 0; i < 9; i++) {
    for (int j = 0; j < 9 - i; j++) {
        if (arr[j] > arr[j + 1]) {
            // Swap arr[j] and arr[j+1]
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

```
// Print sorted array
```

```
printf("Sorted array in ascending order:\n");
for (int i = 0; i < 10; i++) {
    printf("%d ", arr[i]);
}
printf("\n");
```

```
return 0;
```

LAB EXERCISE 2: Matrix Addition • Write a C program that accepts two 2x2 matrices from the user and adds them. Display the resultant matrix. • Challenge: Extend the program to work with 3x3 matrices and matrix multiplication.

```
#include<conio.h>
#include<stdio.h>
int main()
{
    int matrix1[2][2];
    int matrix2[2][2];
    int resultMatrix[2][2];

    // Input for matrix1
    printf("Enter elements of first 2x2 matrix:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("Enter element at position (%d,%d): ", i, j);
            scanf("%d", &matrix1[i][j]);
        }
    }

    // Input for matrix2
    printf("Enter elements of second 2x2 matrix:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("Enter element at position (%d,%d): ", i, j);
            scanf("%d", &matrix2[i][j]);
        }
    }

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            resultMatrix[i][j] = matrix1[i][j] + matrix2[i][j];
        }
    }
}
```

```
    printf("\nResultant Matrix (Sum of the two matrices):\n");  
    for (int i = 0; i < 2; i++) {  
        for (int j = 0; j < 2; j++) {  
            printf("%d\t", resultMatrix[i][j]);  
        }  
        printf("\n");  
    }  
}
```

LAB EXERCISE 3: Sum of Array Elements • Write a C program that takes N numbers from the user and stores them in an array. The program should then calculate and display the sum of all array elements. • Challenge: Modify the program to also find the average of the numbers.

```
#include <stdio.h>
#include <conio.h>

int main() {
    int size;
    int sum = 0;
    float avg;

    printf("Enter the size of the array: ");
    scanf("%d", &size);
    int arr[size];

    printf("Enter %d elements:\n", size);
    for (int i = 0; i < size; i++) {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &arr[i]);
    }

    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    avg = sum / size;

    // Print the sum
    printf("Sum of array elements: %d\n", sum);
    printf("the avg is %f", avg);

    return 0;
```

5. Functions

LAB EXERCISE 1: Fibonacci Sequence • Write a C program that generates the Fibonacci sequence up to N terms using a recursive function. • Challenge: Modify the program to calculate the Nth Fibonacci number using both iterative and recursive methods. Compare their efficiency.

Recursive Method

```
#include <stdio.h>
#include <conio.h>

int Fibonacci(int n){

    if (n <= 1){
        return n;
    }

    return Fibonacci(n - 1) + Fibonacci(n - 2);
}

int main(){
    int n;
    printf("enter the range:");
    scanf("%d",&n);
    for (int i = 0; i < n; i++)
    {
        printf(" %d ", Fibonacci(i));
    }

    return 0;
}
```

Iterative method:

```
#include<stdio.h>
#include<conio.h>
```

```
int fibonacci(int f)
{
```

```
    if(f==0)
    {
        return 0;
    }
```

```
    if(f==1)
    {
        return 1;
    }
```

```
    int a=0,b=1,temp;
    for(int i=2;i<=f;i++)
    {
        temp=a+b;
        a=b;
        b=temp;
    }
    return b;
```

```
}
```

```
int main()
```

```
{
    int n;
    printf("Enter the range : ");
    scanf("%d", &n);
```

```

    for(int i=0;i<n;i++)
    {
```

```
        printf(" %d ",fibonacci(i));  
    }  
  
    return 0;  
}
```

Conclusion:

The iterative approach (using loops) is significantly more efficient for calculating Fibonacci numbers than the naive recursive approach because it avoids redundant calculations and has a much lower time complexity, specifically $O(n)$ compared to the naive recursive method's $O(2^n)$. The naive recursive method has exponential time complexity due to repeatedly recalculating the same Fibonacci numbers, while the loop-based method builds the sequence from the bottom up, solving each subproblem only once.

LAB EXERCISE 2: Factorial Calculation • Write a C program that calculates the factorial of a given number using a function. • Challenge: Implement both an iterative and a recursive version of the factorial function and compare their performance for large numbers.

```
#include <stdio.h>
#include <time.h>

// Iterative factorial function
unsigned long long factorial_iterative(int n) {
    unsigned long long result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

// Recursive factorial function
unsigned long long factorial_recursive(int n) {
    if (n <= 1)
        return 1;
    else
        return n * factorial_recursive(n - 1);
}

int main() {
    int number;
    printf("Enter a non-negative integer: ");
    scanf("%d", &number);

    if (number < 0) {
        printf("Factorial is not defined for negative numbers.\n");
        return 1;
    }

    clock_t start, end;
    double time_iterative, time_recursive;
```

```
start = clock();
unsigned long long fact_iter = factorial_iterative(number);
end = clock();
time_iterative = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
start = clock();
unsigned long long fact_rec = factorial_recursive(number);
end = clock();
time_recursive = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
/
```

```
printf("Iterative factorial of %d is: %llu\n", number, fact_iter);
printf("Time taken by iterative method: %f seconds\n", time_iterative);
```

```
printf("Recursive factorial of %d is: %llu\n", number, fact_rec);
printf("Time taken by recursive method: %f seconds\n", time_recursive);
```

```
return 0;
```

```
}
```

LAB EXERCISE 3: Palindrome Check • Write a C program that takes a number as input and checks whether it is a palindrome using a function. • **Challenge:** Modify the program to check if a given string is a palindrome.

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

// Function to check if a string is palindrome
bool isStringPalindrome(char str[]) {
    int left = 0;
    int right = strlen(str) - 1;

    while (left < right) {
        if (str[left] != str[right]) {
            return false;
        }
        left++;
        right--;
    }
    return true;
}

int main() {
    char str[100];
    printf("Enter a string: ");
    // Read string with spaces
    fgets(str, sizeof(str), stdin);

    // Remove trailing newline character if present
    size_t len = strlen(str);
    if (len > 0 && str[len - 1] == '\n') {
        str[len - 1] = '\0';
    }

    if (isStringPalindrome(str)) {
        printf("\"%s\" is a palindrome string.\n", str);
    } else {
        printf("\"%s\" is not a palindrome string.\n", str);
    }
}
```

```
}
```

```
return 0;
```

```
}
```

6. Strings

LAB EXERCISE 1: String Reversal • Write a C program that takes a string as input and reverses it using a function. • Challenge: Write the program without using built-in string handling functions.

```
#include <stdio.h>
```

```
// Function to calculate string length without using built-in functions
```

```
int stringLength(char str[]) {  
    int length = 0;  
    while (str[length] != '\0') {  
        length++;  
    }  
    return length;  
}
```

```
// Function to reverse the string in place
```

```
void reverseString(char str[]) {  
    int start = 0;  
    int end = stringLength(str) - 1;  
    while (start < end) {  
        // Swap characters  
        char temp = str[start];  
        str[start] = str[end];  
        str[end] = temp;  
  
        start++;  
        end--;  
    }  
}
```

```
int main() {  
    char str[100];
```

```
printf("Enter a string: ");
// Read string with spaces
fgets(str, sizeof(str), stdin);

// Remove trailing newline character if present
int i = 0;
while (str[i] != '\0') {
    if (str[i] == '\n') {
        str[i] = '\0';
        break;
    }
    i++;
}

reverseString(str);

printf("Reversed string: %s\n", str);

return 0;
}
```

LAB EXERCISE 2: Count Vowels and Consonants • Write a C program that takes a string from the user and counts the number of vowels and consonants in the string. • Challenge: Extend the program to also count digits and special characters.

```
#include <stdio.h>
#include <conio.h>
```

```
int main() {
    char str[100];
    int vowels = 0, consonants = 0, digits = 0, specialChars = 0;

    printf("Enter a string: ");
    // Read string with spaces
    fgets(str, sizeof(str), stdin);

    // Iterate through each character until null terminator
    for (int i = 0; str[i] != '\0'; i++) {
        char ch = str[i];

        if (ch == '\n') {
            continue;
        }

        if (isalpha(ch)) {
            ch = tolower(ch);
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
                vowels++;
            } else {
                consonants++;
            }
        } else if (isdigit(ch)) {
            digits++;
        } else {
            specialChars++;
        }
    }

    printf("Vowels: %d, Consonants: %d, Digits: %d, Special Characters: %d\n", vowels, consonants, digits, specialChars);
}
```

```
        // Count as special character if not space
        if (!isspace(ch)) {
            specialChars++;
        }
    }
}

printf("Vowels: %d\n", vowels);
printf("Consonants: %d\n", consonants);
printf("Digits: %d\n", digits);
printf("Special characters: %d\n", specialChars);

return 0;
}
```

LAB EXERCISE 3: Word Count • Write a C program that counts the number of words in a sentence entered by the user. • **Challenge:** Modify the program to find the longest word in the sentence.

Extra Logic Building Challenges

Lab Challenge 1: Armstrong Number • Write a C program that checks whether a given number is an Armstrong number or not (e.g., $153 = 1^3 + 5^3 + 3^3$). • Challenge: Write a program to find all Armstrong numbers between 1 and 1000.

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int countDigits(int num) {
```

```
    int count = 0;
```

```
    while (num != 0) {
```

```
        count++;
```

```
        num /= 10;
```

```
    }
```

```
    return count;
```

```
int isArmstrong(int num) {
```

```
    int original = num;
```

```
    int digits = countDigits(num);
```

```
    int sum = 0;
```

```
    while (num != 0) {
```

```
        int digit = num % 10;
```

```
        sum += (int)pow(digit, digits);
```

```
        num /= 10;
```

```
    }
```

```
    return (sum == original);
```

```
}
```

```
int main() {
```

```
    int number;
```

```
printf("Enter a number to check if it is an Armstrong number: ");  
scanf("%d", &number);
```

```
if (isArmstrong(number)) {  
    printf("%d is an Armstrong number.\n", number);  
} else {  
    printf("%d is not an Armstrong number.\n", number);  
}
```

```
printf("\nArmstrong numbers between 1 and 1000 are:\n");  
for (int i = 1; i <= 1000; i++) {  
    if (isArmstrong(i)) {  
        printf("%d ", i);  
    }  
}  
printf("\n");
```

```
return 0;  
}
```

Lab Challenge 2: Pascal's Triangle • Write a C program that generates Pascal's Triangle up to N rows using loops. • Challenge: Implement the same program using a recursive function.

```
#include <stdio.h>
#include <conio.h>

int main() {

    int n = 5;

    // Outer loop to print each row
    for (int i = 0; i < n; i++) {

        // Print leading spaces for alignment
        for (int j = 0; j < n - i - 1; j++)
            printf(" ");

        // Print values in row
        int val = 1;
        for (int k = 0; k <= i; k++) {
            printf("%d ", val);

            // Calculate the next value in the row
            val = val * (i - k) / (k + 1);
        }
        printf("\n");
    }

    return 0;
}
```

Lab Challenge 3: Number Guessing Game • Write a C program that implements a simple number guessing game. The program should generate a random number between 1 and 100, and the user should guess the number within a limited number of attempts. • Challenge: Provide hints to the user if the guessed number is too high or too low.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int secretNumber, guess;
    int maxAttempts = 7; // You can adjust the number of attempts
    int attempts = 0;

    // Initialize random number generator
    srand(time(NULL));
    secretNumber = (rand() % 100) + 1; // Random number between 1
    and 100

    printf("Welcome to the Number Guessing Game!\n");
    printf("I have selected a number between 1 and 100.\n");
    printf("You have %d attempts to guess it.\n", maxAttempts);

    while (attempts < maxAttempts) {
        printf("Enter your guess: ");
        if (scanf("%d", &guess) != 1) {
            // Handle invalid input
            printf("Invalid input. Please enter an integer.\n");
            // Clear input buffer
            while (getchar() != '\n');
            continue;
        }
    }
```

```
    attempts++;

    if (guess == secretNumber) {
        printf("Congratulations! You guessed the number in %d
attempt%s.\n", attempts, attempts == 1 ? "" : "s");
        break;
    } else if (guess < secretNumber) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too high! Try again.\n");
    }
}

if (attempts == maxAttempts && guess != secretNumber) {
    printf("Sorry, you've used all your attempts. The number was
%d.\n", secretNumber);
}

return 0;
}
```

